

Homework 7: Combined Team Report

When Your Startup's Flash Sale Almost Failed
CS 6650 - Distributed Systems

Team Members

Mayank Tamakuwala | Parthav Nuthalapati | Yatish Wutla

Individual Contributions

Each team member independently built and deployed the full infrastructure (Terraform, Go application, ECS services, SNS/SQS, Lambda) and ran all load tests on separate AWS accounts. This combined report compares and contrasts the results from all three implementations.

- Mayank Tamakuwala: Full implementation with 15-slot buffered channel for sync concurrency. Deployed on AWS Innovation Sandbox (us-west-2). Used Terraform with `enable_lambda` toggle to safely separate Part II and Part III deployments.
- Parthav Nuthalapati: Full implementation with single-slot buffered channel (concurrency = 1). Deployed on AWS Academy (us-east-1). Observed extreme sync bottleneck with 56-second response times.
- Yatish Wutla: Full implementation with single-slot buffered channel (concurrency = 1). Deployed on AWS Academy (us-east-1). Observed 60% failure rate under sync flash sale due to timeouts.

Part II: Synchronous vs Asynchronous Order Processing

Implementation Differences

All three team members used the same core architecture: a Go application deployed on ECS Fargate with SNS for event publishing and SQS for background processing. The key implementation difference was the sync concurrency setting:

Member	Sync Concurrency	Region
Mayank	15 buffered channel slots	us-west-2
Parthav	1 buffered channel slot	us-east-1
Yatish	1 buffered channel slot	us-east-1

This difference in concurrency settings led to significantly different sync behavior: Mayank's system could process up to 5 orders/second synchronously (15 slots / 3s delay), while Parthav's and Yatish's systems were limited to 0.33 orders/second (1 slot / 3s delay). This created a more dramatic contrast between sync and async in Parthav's and Yatish's experiments.

Phase 1: Sync Normal Operations (5 users, 30 seconds)

Metric	Mayank	Parthav	Yatish
Total Requests	33	9	16
Failure Rate	0%	0%	0%
Avg Response Time	3,168 ms	10,478 ms	12,320 ms
P95	3,300 ms	15,000 ms	15,000 ms
Throughput	1.25 req/s	0.4 req/s	0.4 req/s

All three systems achieved 0% failure rate under normal load, confirming the system works correctly at low traffic. However, the throughput and latency differ significantly due to concurrency settings. Mayank's 15-slot configuration allowed multiple concurrent payments, resulting in ~3.2s response times (essentially just the payment delay). Parthav's and Yatish's single-slot configurations forced all 5 users to queue behind one payment slot, causing response times to climb to 10-15 seconds as users waited their turn.

Phase 2: Sync Flash Sale (20 users, 60 seconds)

Metric	Mayank	Parthav	Yatish
Total Requests	285	19	40
Failure Rate	0%	0%	60%
Avg Response Time	3,720 ms	29,631 ms	57,346 ms
P95	4,900 ms	56,000 ms	60,000 ms
Throughput	4.85 req/s	0.4 req/s	0.3 req/s

Bottleneck Analysis:

The flash sale test revealed dramatically different failure modes depending on concurrency:

Mayank's system (15 slots): No requests failed, but latency increased from 3.2s to 3.7s as some requests queued behind the 15-slot buffer. The system handled the load gracefully but was approaching its throughput ceiling of 5 orders/second.

Parthav's system (1 slot): No requests technically failed, but with a single payment slot, response times climbed to 30-56 seconds. The system processed only ~1.6% of demanded load. Customers waited nearly a minute per order.

Yatish's system (1 slot): 60% of requests timed out after 60 seconds waiting for the single payment slot. Only 40 requests were attempted and 23 failed. The system was completely overwhelmed, demonstrating the most severe sync bottleneck.

These results illustrate that increasing sync concurrency helps, but fundamentally cannot solve the problem. Even Mayank's 15-slot system would collapse at true flash sale scale (60 orders/sec).

Phase 3: Async Flash Sale (20 users, 60 seconds)

Metric	Mayank	Parthav	Yatish
Total Requests	2,727	3,329	2,142
Failure Rate	0%	0%	0%
Avg Response Time	132 ms	51 ms	45 ms
Median	110 ms	48 ms	42 ms
P95	230 ms	66 ms	66 ms
Throughput	45.70 req/s	56.7 req/s	57.9 req/s

Switching to async processing produced consistent results across all three implementations: 0% failure rate and sub-200ms response times. The API simply publishes to SNS and returns 202 Accepted immediately, decoupling order acceptance from payment processing.

The throughput variation (45-58 req/s) can be attributed to regional latency differences (us-west-2 vs us-east-1) and Locust client-side timing. The key takeaway is that all three systems accepted orders at rates 9x to 175x higher than their respective sync implementations.

Sync vs Async Improvement Ratio:

Member	Sync Orders	Async Orders	Improvement
Mayank	285	2,727	9.6x
Parthav	19	3,329	175x
Yatish	40 (60% failed)	2,142	~150-300x

Phase 4: Queue Analysis

Metric	Mayank	Parthav	Yatish
Acceptance rate	~46 ord/s	~57 ord/s	~57 ord/s
1-worker rate	0.33 ord/s	0.33 ord/s	0.33 ord/s
Queue growth rate	~45 msg/s	~56 msg/s	~57 msg/s
Peak backlog	~5,260	~1,780	~3,400
1-worker clear time	~136 min	~90 min	~170 min

All three systems experienced rapid queue buildup with a single worker. The queue growth rate was nearly identical to the acceptance rate since a single worker (0.33 orders/sec) is negligible compared to the incoming ~50+ orders/sec. The peak backlog differences reflect the total number of orders accepted during each test.

Phase 5: Worker Scaling Comparison

Each member scaled the WORKER_COUNT goroutines within a single ECS Fargate task (256 CPU, 512 MB) and re-ran the async flash sale test.

Mayank's Results:

Workers	Rate (ord/s)	Reqs	Clear Time
1	0.33	2,727	~136 min
5	1.67	2,880	~29 min
20	6.67	2,832	~7 min
100	33.33	2,829	~1.4 min

Parthav's Results:

Workers	Rate (ord/s)	Reqs	Queue Behavior
1	0.33	3,329	Never drains
5	1.67	3,310	Drains slowly

20	6.67	3,515	Still growing
100	33.33	3,588	Drained in ~15 min

Yatish's Results:

Workers	Rate (ord/s)	Reqs	Queue Behavior
1	0.33	2,142	Never drains
5	1.67	4,329	Builds, slow drain
20	6.67	6,249	Spike, drains after
100	33.33	7,889	Near zero, quick drain

Minimum Workers to Prevent Queue Buildup:

All three members arrived at a similar calculation. With an acceptance rate of ~47-57 orders/second and each worker processing 1 order per 3 seconds:

- Mayank: $47 \times 3 = 141$ workers needed
- Parthav: $57 \times 3 = 171$ workers needed
- Yatish: $57 \times 3 = 171$ workers needed

In practice, 100 workers were sufficient because the flash sale is time-limited (60 seconds), so workers can catch up after the spike ends. Yatish observed that with 100 workers, the queue stayed near zero and drained quickly. Parthav noted full drain in ~15 minutes.

Analysis Questions

How many times more orders did async accept vs sync?

The improvement factor varied based on sync concurrency settings:

- Mayank (15 slots): 9.6x more orders (2,727 vs 285)
- Parthav (1 slot): 175x more orders (3,329 vs 19)
- Yatish (1 slot): 150-300x more orders (2,142 vs ~8 successful)

The async approach eliminated the dependency on payment processing speed entirely. Regardless of sync concurrency settings, all three async implementations achieved similar throughput (~45-58 req/s) since the bottleneck (payment processing) was moved to the background.

What causes queue buildup and how do you prevent it?

Queue buildup occurs when message production rate exceeds consumption rate. All three members observed the same root cause: accepting ~50+ orders/second while processing only 0.33 orders/second per worker. Prevention strategies identified across the team include:

- Scaling worker goroutines within the ECS task (tested by all three members)
- Scaling the number of ECS tasks horizontally
- Setting CloudWatch alarms on ApproximateNumberOfMessagesVisible to trigger auto-scaling
- Switching to Lambda which scales automatically per message (explored in Part III)

When would you choose sync vs async in production?

The team consensus is:

- Sync: when the operation is fast (sub-second), the client needs an immediate definitive result (login, balance check, inventory query), or data consistency must be confirmed before responding.
- Async: when the operation is slow or unpredictable (payment verification, email sending, report generation), the system must remain responsive under traffic spikes, or eventual consistency is acceptable - the customer just needs confirmation that the order was received, not that payment was already processed.

Part III: Lambda - What If You Didn't Need Queues?

Architecture Change

All three members deployed a Lambda function subscribing directly to the SNS topic, replacing the SQS + ECS worker architecture:

Part II: Order API -> SNS -> SQS -> ECS Workers (manually managed)

Part III: Order API -> SNS -> Lambda (AWS managed)

Each Lambda function used the same 3-second payment delay, 512 MB memory, Go runtime (provided.al2), and was triggered directly by SNS. Test orders were sent manually via curl (5-10 orders each) to avoid triggering account deactivation from high Lambda concurrency.

Cold Start Comparison

Metric	Mayank	Parthav	Yatish
Orders sent	10	5-10	8
Cold starts observed	2 of 10	1st request	All 8 (initial)
Avg Init Duration	70-75 ms	44 ms	76 ms
Warm Duration	~3,001 ms	~3,001 ms	~3,001 ms
Cold Billed Duration	~3,075 ms	~3,051 ms	~3,077 ms
Overhead	~2.5%	~1.5%	~2.5%
Memory Used	19-20 MB	22 MB	20 MB

Cold start behavior was consistent across all three implementations. Init Duration ranged from 44-105ms, representing a 1.5-3% overhead on the 3-second payment processing operation. All members observed that cold starts occurred on the first invocation and when Lambda provisioned additional concurrent execution environments. Yatish noted that all 8 of the initial orders triggered cold starts because they arrived close enough together that Lambda spun up 8 separate instances. After ~5-6 minutes of idle time, subsequent invocations would cold start again.

All three members concluded that the cold start overhead is negligible for this workload. It would be more concerning for latency-sensitive sub-100ms operations.

Cost Analysis (Team Consensus)

	ECS (Part II)	Lambda (Part III)
Monthly cost (10K orders)	\$17	\$0 (free tier)
Free until	N/A (always \$17)	~267K orders/month
Break-even with ECS	N/A	~680K-1.7M orders/month
Scales automatically	No	Yes
Always running	Yes	No (pay per use)

All three members calculated the same result: for 10,000 orders/month at 3 seconds and 512 MB, Lambda is completely free under the AWS free tier. The free tier covers up to ~267,000 orders/month before any charges apply. ECS costs \$17/month regardless of whether any orders are processed.

Team Recommendation: Should the Startup Switch to Lambda?

All three team members independently reached the same conclusion: a startup processing under 267K orders/month should switch to Lambda. The reasoning was consistent across the team:

Lambda eliminates all operational overhead: no SQS queue depth monitoring, no manual worker scaling, no ECS health checks, no 3am alerts. It costs \$0 compared to \$17/month for ECS. The 2-3% cold start overhead is negligible on 3-second payment processing. AWS automatically scales Lambda to handle any number of concurrent orders without configuration.

The trade-off acknowledged by all three members is the loss of SQS durability guarantees. SNS retries only twice before discarding a message, whereas SQS retains messages for up to 4 days. There is no built-in dead-letter queue or batch processing capability. For a cost-conscious startup at moderate volume, this trade-off is acceptable.

As order volume grows beyond 267K/month or reliability requirements increase, the team recommends a hybrid approach (SNS -> SQS -> Lambda) that combines the auto-scaling of serverless with the durability of queued processing.